Assignment 06

Question 01

In [12]:
```
import math
t=[1,2,3,4,5]
A=[225,215,250,225,205]
B=[220,225,250,250,210]
r=0.0433
# For investment A
p=[]
for i in range(len(t)):
    pv=A[i]/math.exp(r*t[i])
    p.append(pv)
print(f'Present Value of Investment A: {sum(p)}')

# For investment B
q=[]
for i in range(len(t)):
    pv=B[i]/math.exp(r*t[i])
    q.append(pv)
print(f'Present Value of Investment B: {sum(q)}')
if sum(p)>sum(q):
    print('Investment A is preferable.')
else:
    print('Investment B is preferable.')
```

```
Present Value of Investment A: 986.4869358093922
Present Value of Investment B: 1015.9201516869825
Investment B is preferable.
```

Question 02

In [13]:
```
import numpy as np
F = 100      # Face value
c = 0.08     # Coupon rate
y = 0.11     # Yield
n = 5        # Number of periods
# Cash flows
t = np.arange(1, n+1)
CF = np.full(n, c*F)
CF[-1] += F  # add principal to last cash flow

# Price and duration
P = np.sum(CF * np.exp(-y*t))
D = np.sum(t * CF * np.exp(-y*t)) / P

# Approximate price change for a small yield change
dy = -0.002
dP_approx= -P*D*dy

# Actual price change
```

```
y2 = 0.108
P2 = np.sum(CF * np.exp(-y2*t))
dP_actual = P2 - P

# Print results
print("Bond Price :", P)
print("Duration D:", D)
print("Approx. price change :", dP_approx)
print("Actual price change :", dP_actual)
```

```
Bond Price : 86.80108422328516
Duration D: 4.255974563427866
Approx. price change : 0.738846413064523
Actual price change : 0.7423071720569538
```

Question 03

In [14]:
```
t=[1,2,3,4,5,6]
A=[465,233,632,365,334,248]
r=0.045
n=4
p=[]
for i in range(len(t)):
    pv=A[i]/(1+r/n)**(n*t[i])
    p.append(pv)
print(f'Present value of the investment: {sum(p)}')
```

```
Present value of the investment: 1972.1318910389198
```

Question 04

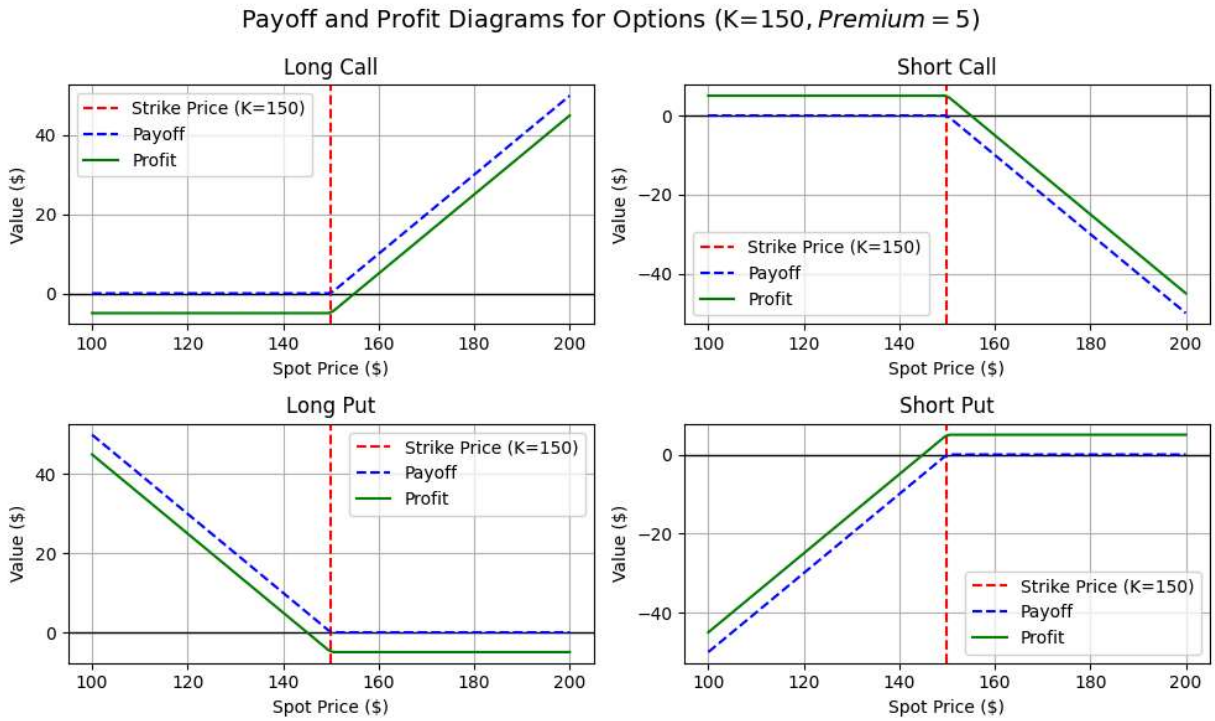
In [15]:
```
import math
# Given data
S0=40     # Initial stock price
r=0.10    # Risk-free rate
T=1       # Time to maturity

# (a) Forward price and initial value
F0=S0*math.exp(r*T)
initial_value=0

print("(a)")
print(f"Forward price (F0): {F0:.2f}")
print(f"Initial value of forward contract: {initial_value:.2f}")

# (b) Six months later
t=0.5         # 6 months
St=45         # Stock price at t = 0.5
Ft=St*math.exp(r*(T-t))  # New forward price
# Value of original forward contract
ft=St-F0*math.exp(-r*(T-t))

print("\n(b)")
print(f"New forward price (Ft): {Ft:.2f}")
print(f"Value of original forward contract (ft): {ft:.2f}")
```

```
(a)
Forward price (F0): 44.21
Initial value of forward contract: 0.00

(b)
New forward price (Ft): 47.31
Value of original forward contract (ft): 2.95
```

Question 05

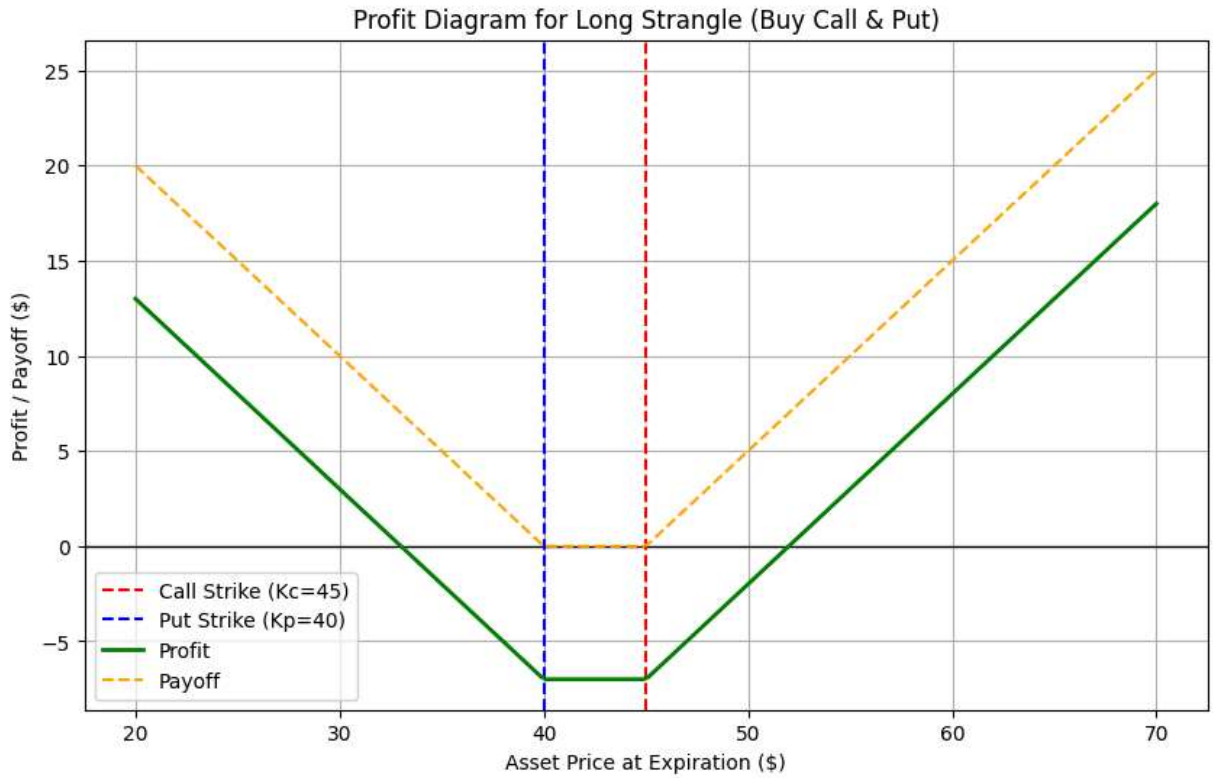
In [16]:

```python
import numpy as np
import matplotlib.pyplot as plt
K=150 # Strike Price
premium=5 # Option premium
S=np.linspace(100,200,100) # Spot price range
# Payoff and Profit calculations
# Long Call
long_call_payoff=np.maximum(S-K,0)
long_call_profit=long_call_payoff-premium
# Short Call
short_call_payoff=-long_call_payoff
short_call_profit=short_call_payoff+premium
# Long Put
long_put_payoff=np.maximum(K-S,0)
long_put_profit=long_put_payoff-premium
# Short Put
short_put_payoff= -long_put_payoff
short_put_profit=short_put_payoff+premium
# Plotting all in subplots
fig,axs=plt.subplots(2,2,figsize=(10,6))
fig.suptitle('Payoff and Profit Diagrams for Options (K=$150, Premium=$5)',fontsize=1
# Titles for each subplot
titles=['Long Call','Short Call','Long Put','Short Put']
payoffs=[long_call_payoff, short_call_payoff, long_put_payoff, short_put_payoff]
profits=[long_call_profit, short_call_profit, long_put_profit, short_put_profit]
# Plot each
for ax, title, payoff, profit in zip(axs.flat, titles, payoffs, profits):
    ax.axhline(0, color='black', linewidth=1)
    ax.axvline(K, color='red', linestyle='--', label=f'Strike Price (K={K})')
    ax.plot(S, payoff, '--', color='blue', label='Payoff')
    ax.plot(S, profit, color='green', label='Profit')
    ax.set_title(title)
    ax.set_xlabel('Spot Price ($)')
    ax.set_ylabel('Value ($)')
    ax.legend()
    ax.grid()

plt.tight_layout()
plt.show()
```



Question 06

In [17]:
```
import numpy as np
import matplotlib.pyplot as plt
K_call=45
K_put=40
call_premium=3
put_premium=4
total_premium=call_premium+put_premium
S = np.linspace(20, 70, 200) # Asset price range
# Payoff calculations
call_payoff=np.maximum(S-K_call,0)
put_payoff=np.maximum(K_put-S,0)
total_payoff=call_payoff+put_payoff
profit=total_payoff-total_premium
# Plotting
plt.figure(figsize=(10,6))
plt.axhline(0, color='black', linewidth=1)
plt.axvline(K_call, color='red', linestyle='--', label=f'Call Strike (Kc={K_call})')
plt.axvline(K_put, color='blue', linestyle='--', label=f'Put Strike (Kp={K_put})')

plt.plot(S, profit, color='green', linewidth=2, label='Profit')
plt.plot(S, total_payoff, color='orange', linestyle='--', label='Payoff')

plt.title('Profit Diagram for Long Strangle (Buy Call & Put)')
plt.xlabel('Asset Price at Expiration ($)')
plt.ylabel('Profit / Payoff ($)')
plt.legend()
plt.grid()
plt.show()
```



Question 07

In [18]:
```
import math
# Given values
S0=19   # Current stock price
K=20    # Strike price
C=1     # Call price
r=0.03  # Risk-free rate
T=4/12  # Time in years (4 months)
# Calculate put price using Put-Call Parity
P=C - S0 + K * math.exp(-r * T)

print(f"The price of the European put option is: ${P:.4f}")
```

```
The price of the European put option is: $1.8010
```

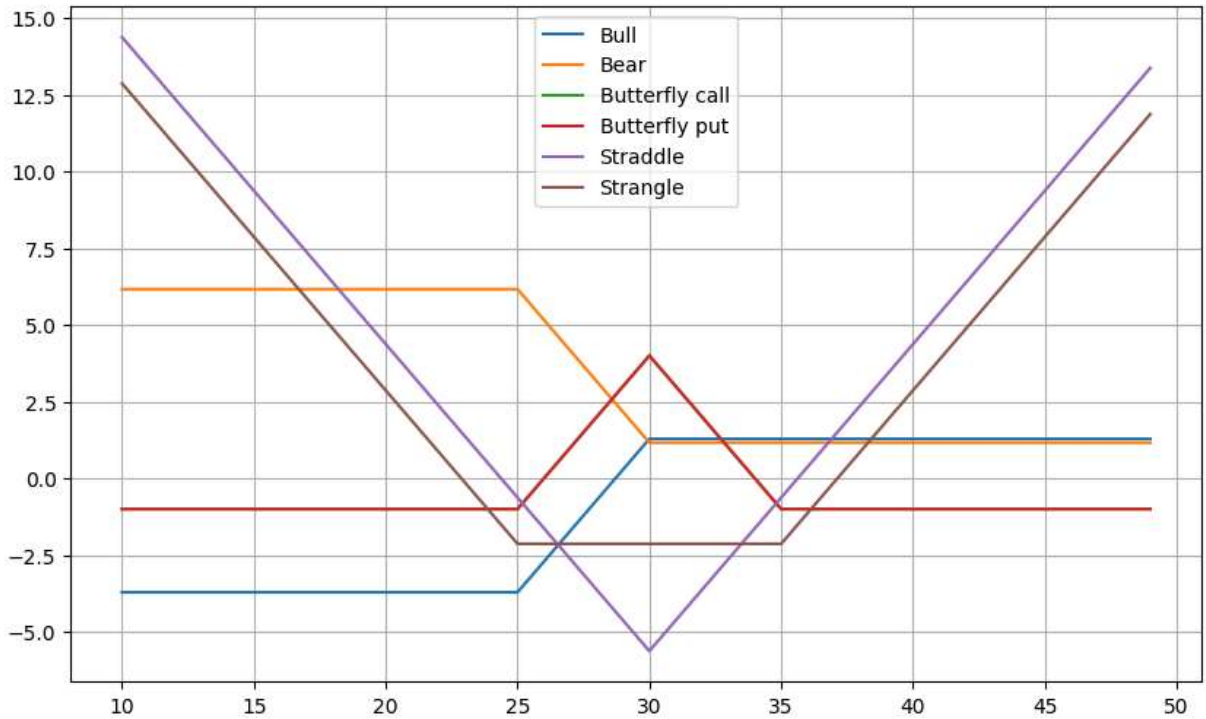
Question 08

In [19]:
```
import numpy as np
import pandas as pd
from math import log,sqrt,exp
from scipy.stats import norm
import matplotlib.pyplot as plt

# Black-Scholes formula for European Call and Put
def bs_price(S,K,r,T,sigma,option_type='call'):
    d1=(log(S/K)+(r+0.5*sigma**2)*T)/(sigma*sqrt(T))
    d2=d1-sigma*sqrt(T)
    if option_type=='call':
        return S*norm.cdf(d1)-K*exp(-r*T)*norm.cdf(d2)
    else:
```

```
        return K*exp(-r*T)*norm.cdf(-d2)-S*norm.cdf(-d1)
# Parameters
S0=32
r=0.05
sigma=0.30
# Strike Prices and Times
K1,K2,K3=25,30,35
T_short=0.5 # 6 months
T_long=1.0  # 1 year

# Compute option prices
call_25_6m=bs_price(S0,K1,r,T_short,sigma,'call')
call_30_6m=bs_price(S0,K2,r,T_short,sigma,'call')
put_25_6m=bs_price(S0,K1,r,T_short,sigma,'put')
put_30_6m=bs_price(S0,K2,r,T_short,sigma,'put')

call_25_1y=bs_price(S0,K1,r,T_long,sigma,'call')
call_30_1y=bs_price(S0,K2,r,T_long,sigma,'call')
call_35_1y=bs_price(S0,K3,r,T_long,sigma,'call')

put_25_1y=bs_price(S0,K1,r,T_long,sigma,'put')
put_30_1y=bs_price(S0,K2,r,T_long,sigma,'put')
put_35_1y=bs_price(S0,K3,r,T_long,sigma,'put')
call_35_6m=bs_price(S0,K3,r,T_short,sigma,'call')

# Display option prices
print("Option Prices:")
print(f"{'Call(25,6m)':<12}: {call_25_6m:.4f}, {'Call(30,6m)':<12}: {call_30_6m:.4f}'
print(f"{'Put(25,6m)':<12}: {put_25_6m:.4f}, {'Put(30,6m)':<12}: {put_30_6m:.4f}")
print(f"{'Call(25,1y)':<12}: {call_25_1y:.4f}, {'Call(30,1y)':<12}: {call_30_1y:.4f},
print(f"{'Put(25,1y)':<12}: {put_25_1y:.4f}, {'Put(30,1y)':<12}: {put_30_1y:.4f}, {'P

# Create stock price range for payoff calculations
S_range=np.arange(10,50,1)
# Strategy functions
def bull_call_spread(S,K1,K2,premium1,premium2):
    payoff=np.maximum(S-K1,0)-np.maximum(S-K2,0)
    cost=premium1-premium2
    return payoff-cost

def bear_put_spread(S,K1,K2,premium1,premium2):
    payoff=np.maximum(K2-S,0)-np.maximum(K1-S,0)
    cost=premium1-premium2
    return payoff-cost

def butterfly_call(S,K1,K2,K3,p1,p2,p3):
    payoff=np.maximum(S-K1,0)-2*np.maximum(S-K2,0)+np.maximum(S-K3,0)
    cost=p1-2*p2+p3
    return payoff-cost

def butterfly_put(S,K1,K2,K3,p1,p2,p3):
    payoff=np.maximum(K1-S,0)-2*np.maximum(K2-S,0)+np.maximum(K3-S,0)
    cost=p1-2*p2+p3
    return payoff-cost

def straddle(S,K,call_premium,put_premium):
```

```
    payoff=np.maximum(S-K,0)+np.maximum(K-S,0)
    cost=call_premium+put_premium
    return payoff-cost

def strangle(S,K1,K2,call_premium,put_premium):
    payoff=np.maximum(S-K2,0)+np.maximum(K1-S,0)
    cost=call_premium+put_premium
    return payoff-cost

# Calculate profit tables
bull=bull_call_spread(S_range,25,30,call_25_6m,call_30_6m)
bear=bear_put_spread(S_range,25,30,put_25_6m,put_30_6m)
butterfly_c=butterfly_call(S_range,25,30,35,call_25_1y,call_30_1y,call_35_1y)
butterfly_p=butterfly_put(S_range,25,30,35,put_25_1y,put_30_1y,put_35_1y)
straddle_profit=straddle(S_range,30,call_30_6m,put_30_6m)
strangle_profit=strangle(S_range,25,35,call_35_6m,put_25_6m)  # Strangle K1=25(put),

# Create tables
df=pd.DataFrame({
    'Stock Price': S_range,
    'Bull Call Spread': bull,
    'Bear Put Spread': bear,
    'Butterfly Call': butterfly_c,
    'Butterfly Put': butterfly_p,
    'Straddle': straddle_profit,
    'Strangle': strangle_profit
})
print("\nProfit Table:")
print(df.head(15))

plt.figure(figsize=(10,6))
plt.plot(S_range,bull,label='Bull')
plt.plot(S_range,bear,label='Bear')
plt.plot(S_range,butterfly_c,label='Butterfly call')
plt.plot(S_range,butterfly_p,label='Butterfly put')
plt.plot(S_range,straddle_profit,label='Straddle')
plt.plot(S_range,strangle_profit,label='Strangle')
plt.legend()
plt.grid()
plt.show()
```

```
Option Prices:
Call(25,6m) : 7.8967, Call(30,6m) : 4.1837
Put(25,6m)  : 0.2794, Put(30,6m)  : 1.4430
Call(25,1y) : 8.9176, Call(30,1y) : 5.5982, Call(35,1y) : 3.2797
Put(25,1y)  : 0.6984, Put(30,1y)  : 2.1351, Put(35,1y)  : 4.5728

Profit Table:
    Stock Price  Bull Call Spread  Bear Put Spread  Butterfly Call  \
0            10         -3.712982         6.163568        -1.00091
1            11         -3.712982         6.163568        -1.00091
2            12         -3.712982         6.163568        -1.00091
3            13         -3.712982         6.163568        -1.00091
4            14         -3.712982         6.163568        -1.00091
5            15         -3.712982         6.163568        -1.00091
6            16         -3.712982         6.163568        -1.00091
7            17         -3.712982         6.163568        -1.00091
8            18         -3.712982         6.163568        -1.00091
9            19         -3.712982         6.163568        -1.00091
10           20         -3.712982         6.163568        -1.00091
11           21         -3.712982         6.163568        -1.00091
12           22         -3.712982         6.163568        -1.00091
13           23         -3.712982         6.163568        -1.00091
14           24         -3.712982         6.163568        -1.00091

    Butterfly Put   Straddle   Strangle
0        -1.00091  14.373279  12.867196
1        -1.00091  13.373279  11.867196
2        -1.00091  12.373279  10.867196
3        -1.00091  11.373279   9.867196
4        -1.00091  10.373279   8.867196
5        -1.00091   9.373279   7.867196
6        -1.00091   8.373279   6.867196
7        -1.00091   7.373279   5.867196
8        -1.00091   6.373279   4.867196
9        -1.00091   5.373279   3.867196
10       -1.00091   4.373279   2.867196
11       -1.00091   3.373279   1.867196
12       -1.00091   2.373279   0.867196
13       -1.00091   1.373279  -0.132804
14       -1.00091   0.373279  -1.132804
```



Question 09

In [20]:
```
import math
from scipy.stats import norm
# Given values
S0=30
K=29
r=0.05
sigma=0.25
T=4/12  # 4 months

# Black-Scholes formula for European Call and Put
def black_scholes(S,K,r,T,sigma):
    d1=(math.log(S/K)+(r+0.5*sigma**2)*T)/(sigma*math.sqrt(T))
    d2=d1-sigma*math.sqrt(T)
    call_price=S*norm.cdf(d1)-K*math.exp(-r*T)*norm.cdf(d2)
    put_price=K*math.exp(-r*T)*norm.cdf(-d2)-S*norm.cdf(-d1)
    return call_price, put_price
# Calculate prices
call_euro,put_euro=black_scholes(S0,K,r,T,sigma)

# American call on non-dividend stock = European call price
call_american=call_euro
# Check Put-Call Parity: C - P = S - K*e^(-rT)
lhs=call_euro-put_euro
rhs=S0-K*math.exp(-r*T)

print(f"European Call Price: {call_euro:.4f}")
print(f"American Call Price: {call_american:.4f}")
print(f"European Put Price: {put_euro:.4f}")
print("\nPut-Call Parity Check:")
```

```
print(f"LHS (C - P) = {lhs:.4f}, RHS (S0 - K*e^(-rT)) = {rhs:.4f}")
print("Parity holds?" , abs(lhs - rhs) < 1e-6)
```

```
European Call Price: 2.5251
American Call Price: 2.5251
European Put Price: 1.0458

Put-Call Parity Check:
LHS (C - P) = 1.4793, RHS (S0 - K*e^(-rT)) = 1.4793
Parity holds? True
```

Question 10

In [21]:

```
import math
# Given data
F0=60       # Futures price
K=60        # Strike price
sigma=0.30  # Volatility
r=0.08      # Risk-free rate
T=0.5       # Time in years
n=2         # Steps
# Step size and factors
dt=T/n
u=math.exp(sigma*math.sqrt(dt))   # Up factor
d=1/u                             # Down factor
p=(math.exp(r*dt)-d)/(u-d)        # Risk-neutral probability

# Futures prices at nodes
Fuu=F0*u*u
Fud=F0*u*d
Fdd=F0*d*d
# Terminal payoffs
Cuu=max(Fuu-K,0)
Cud=max(Fud-K,0)
Cdd=max(Fdd-K,0)

# One step back
Cu=math.exp(-r*dt)*(p*Cuu+(1-p)*Cud)
Cd=math.exp(-r*dt)*(p*Cud+(1-p)*Cdd)
# Root value (European)
C0=math.exp(-r*dt)*(p*Cu+(1-p)*Cd)
# American check at node 1
Cu_early=max(F0*u-K,0)
Cd_early=max(F0*d-K,0)
early_exercise_possible=(Cu_early > Cu) or (Cd_early > Cd)

print(f"European Call Option Price: {C0:.4f}")
print(f"Early exercise for American Call? {'Yes' if early_exercise_possible else 'No'
```

```
European Call Option Price: 5.6580
Early exercise for American Call? No
```

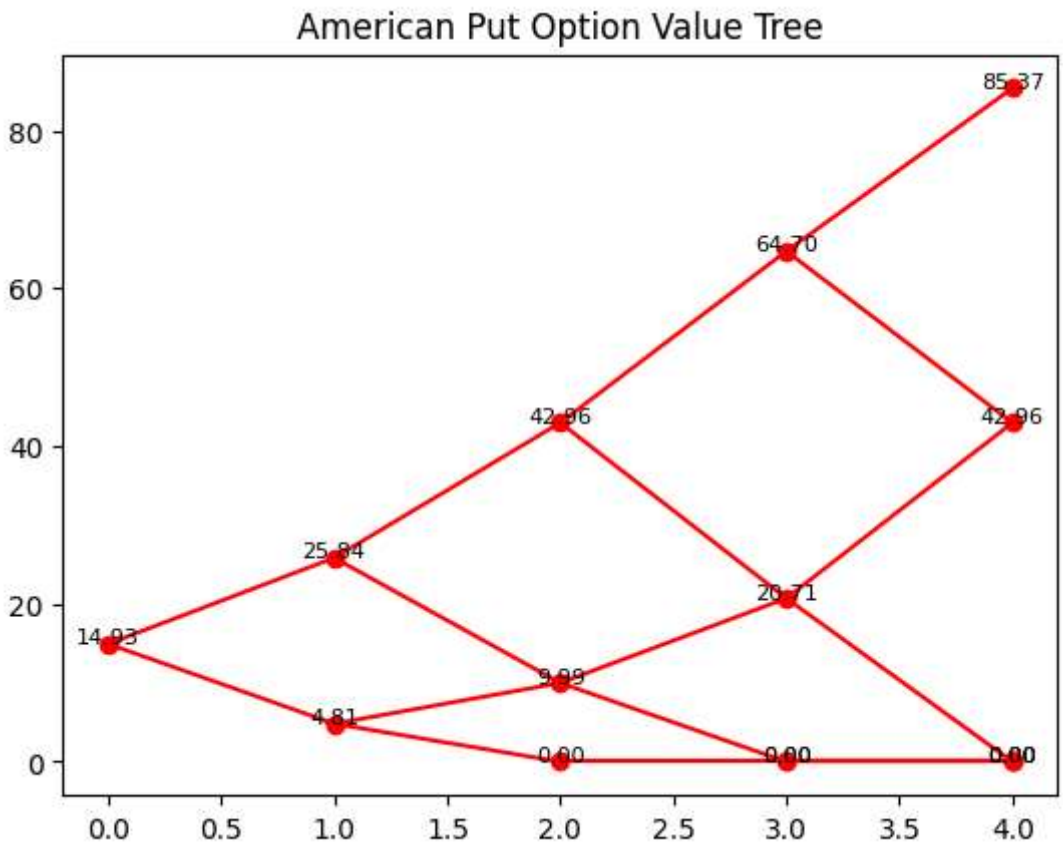
Question 11

In [22]:

```
import numpy as np
import matplotlib.pyplot as plt
```

```
S0,K,r,q,sigma,T,N=484,480,0.1,0.03,0.25,2/12,4
dt=T/N
u=np.exp(sigma*np.sqrt(dt))
d=1/u
p=(np.exp((r-q)*dt)-d)/(u-d)

# Stock & option trees
S=[[S0*u**j*d**(i-j)for j in range(i+1)]for i in range(N+1)]
P=[[0]*(i+1)for i in range(N+1)]
P[-1]=[max(K-s,0)for s in S[-1]]

for i in range(N-1,-1,-1):
  for j in range(i+1):
    cont=np.exp(-r*dt)*(p*P[i+1][j+1]+(1-p)*P[i+1][j])
    P[i][j]=max(K-S[i][j],cont)

print("American put price:",P[0][0])

# --- Plot tree ---
for i in range(N):
  for j in range(i+1):
    plt.plot([i,i+1],[P[i][j],P[i+1][j]],'r-')
    plt.plot([i,i+1],[P[i][j],P[i+1][j+1]],'r-')
for i in range(N+1):
  for j,v in enumerate(P[i]):
    plt.scatter(i,v,c='r')
    plt.text(i,v,f"{v:.2f}",ha='center',fontsize=8)

plt.title("American Put Option Value Tree")
plt.show()
```

```
American put price: 14.933233649496977
```


American Put Option Value Tree
85.37
64.70
42.96
42.96
25.84
20.71
14.93
9.89
4.81
0.00
0.00
0.00
0
20
40
60
80
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0